Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Git Branch

‹ Previous Next ›

What is a Git Branch?

SHARE

In Git, a `branch` is like a separate workspace where you can make changes and try new ideas without affecting the main project. Think of it as a "parallel universe" for your code.

Change Platform:
GitHub
Bitbucket
GitLab

Why Use Branches?

Branches let you work on different parts of a project, like new features or bug fixes, without interfering with the main branch.

Common Reasons to Create a Branch

- Developing a new feature
- Fixing a bug

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Example: With and Without Git

Let's say you have a large project, and you need to update the design on it.

How would that work without and with Git:

Without Git:

- Make copies of all the relevant files to avoid impacting the live version
- Start working with the design and find that code depend on code in other files, that also need to be changed!
- Make copies of the dependant files as well. Making sure that every file dependency references the correct file name
- EMERGENCY! There is an unrelated error somewhere else in the project that needs to be fixed ASAP!
- Save all your files, making a note of the names of the copies you were working on
- Work on the unrelated error and update the code to fix it
- Go back to the design, and finish the work there
- Copy the code or rename the files, so the updated design is on the live version
- (2 weeks later, you realize that the unrelated error was not fixed in the new design version because you copied the files before the fix)

With Git:

- With a new branch called new-design, edit the code directly without impacting the main branch
- EMERGENCY! There is an unrelated error somewhere else in the project that needs to be fixed ASAP!
- Create a new branch from the main project called small-error-fix
- Fix the unrelated error and merge the small-error-fix branch with the main branch
- You go back to the new-design branch, and finish the work there
- Merge the new-design branch with main (getting alerted to the small error fix that you were missing)

Branches allow you to work on different parts of a project without impacting the main branch.

When the work is complete, a branch can be merged with the main project.

W3Schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Creating a New Branch

Let's say you want to add a new feature. You can create a new branch for it.

Let add some new features to our `index.html` page.

We are working in our local repository, and we do not want to disturb or possibly wreck the main project.

So we create a new `branch`:

Example

```
[user@localhost] $ git branch hello-world-images
```

Now we created a new `branch` called "`hello-world-images`"

Listing All Branches

Let's confirm that we have created a new `branch`.

To see all branches in your repository, use:

Example

```
[user@localhost] $ git branch
  hello-world-images
* master
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Switching Between Branches

`checkout` is the command used to check out a `branch`.

Moving us **from** the current `branch`, **to** the one specified at the end of the command:

Example

```
[user@localhost] $ git checkout hello-world-images
Switched to branch 'hello-world-images'
```

Now you can work in your new branch without affecting the main branch.

Working in a Branch

Now we have moved our current workspace from the master branch, to the new `branch`

Open your favourite editor and make some changes.

For this example, we added an image (img_hello_world.jpg) to the working folder and a line of code in the `index.html` file:

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Hello World!</title>
<link rel="stylesheet" href="bluestyle.css">
</head>
<body>
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

```
<p>A new line in our file!</p>

</body>
</html>
```

We have made changes to a file and added a new file in the working directory (same directory as the `main` `branch`).

Now check the status of the current `branch`:

Example

```
[user@localhost] $ git status
                   On branch hello-world-images
                   Changes not staged for commit:
                     (use "git add ..." to update what will be committed)
                     (use "git restore ..." to discard changes in working
                           modified:   index.html

                   Untracked files:
                     (use "git add ..." to include in what will be committ
                           img_hello_world.jpg

                   no changes added to commit (use "git add" and/or "git c
```

So let's go through what happens here:

- There are changes to our index.html, but the file is not staged for `commit`
- `img_hello_world.jpg` is not `tracked`

So we need to add both files to the Staging Environment for this `branch`:

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Using `--all` instead of individual filenames will **Stage** all changed (new, modified, and deleted) files.

Check the `status` of the `branch`:

Example

```
[user@localhost] $ git status
On branch hello-world-images
Changes to be committed:
  (use "git restore --staged ..." to unstage)
    new file: img_hello_world.jpg
    modified: index.html
```

We are happy with our changes. So we will commit them to the `branch`:

Example

```
[user@localhost] $ git commit -m "Added image to Hello World"
[hello-world-images 0312c55] Added image to Hello Worlc
2 files changed, 1 insertion(+)
create mode 100644 img_hello_world.jpg
```

Now we have a new `branch`, that is different from the master `branch`.

Note: Using the `-b` option on `checkout` will create a new branch, and move to it, if it does not exist

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE



REMOVE ADS

Switching Between Branches

Now let's see just how quick and easy it is to work with different branches, and how well it works.

We are currently on the branch `hello-world-images`. We added an image to this branch, so let's list the files in the current directory:

Example

```
[user@localhost] $ ls
README.md  bluestyle.css  img_hello_world.jpg  index.ht
```

We can see the new file `img_hello_world.jpg`, and if we open the html file, we can see the code has been altered. All is as it should be.

Now, let's see what happens when we change branch to `master`

Example

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

The new image is not a part of this branch. List the files in the current directory again:

Example

```
[user@localhost] $ ls
README.md  bluestyle.css  index.html
```



`img_hello_world.jpg` is no longer there! And if we open the html file, we can see the code reverted to what it was before the alteration.

See how easy it is to work with branches? And how this allows you to work on different things?

Emergency Branch

Now imagine that we are not yet done with hello-world-images, but we need to fix an error on master.

I don't want to mess with master directly, and I do not want to mess with hello-world-images, since it is not done yet.

So we create a new branch to deal with the emergency:

Example

```
[user@localhost] $ git checkout -b emergency-fix
Switched to a new branch 'emergency-fix'
```

Now we have created a new branch from master, and changed to it. We can safely fix the error without disturbing the other branches.

Let's fix our imaginary error:

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

```
<!DOCTYPE html>
<html>
<head>
<title>Hello World!</title>
<link rel="stylesheet" href="bluestyle.css">
</head>
<body>

<h1>Hello world!</h1>
<p>This is the first file in my new Git Repo.</p>
<p>This line is here to show how merging works.</p>

</body>
</html>
```

We have made changes in this file, and we need to get those changes to the master branch.

Check the status:

Example

```
[user@localhost] $ git status
On branch emergency-fix
Changes not staged for commit:
  (use "git add ..." to update what will be committed)
  (use "git restore ..." to discard changes in working
        modified:   index.html

no changes added to commit (use "git add" and/or "git c
```

stage the file, and commit:

Example

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

Now we have a fix ready for master, and we need to merge the two branches.

Deleting a Branch

When you're done with a branch, you can delete it:

Example

```
[user@localhost] $ git branch -d hello-world-images
```

This deletes the branch named `hello-world-images` (if it's already merged).

Best Practices for Working with Branches

- Use clear, descriptive branch names (like `feature/login-page` or `bugfix/header-crash`).
- Keep each branch focused on a single purpose or feature.
- Regularly merge changes from the main branch to keep your branch up-to-date.
- Delete branches that are no longer needed to keep your repository clean.

Practical Examples

- **Rename a branch:** `git branch -m old-name new-name`

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

- See which branch you're on: `git status`

Troubleshooting

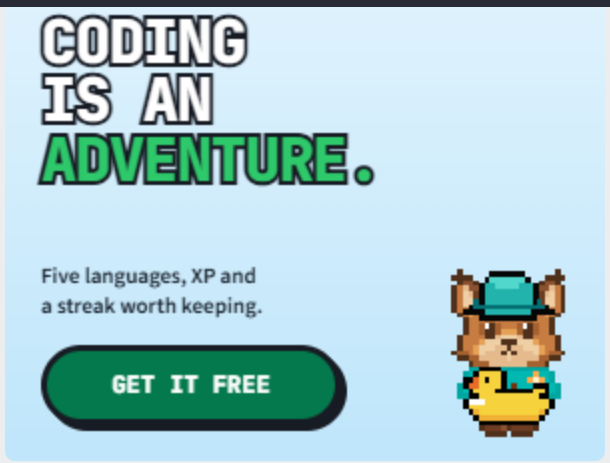
If you don't see your changes on the main branch, remember: changes in one branch stay there until you merge them.

When deleting a branch, make sure it's merged first. If you try to delete an unmerged branch, Git will prevent you from doing so.

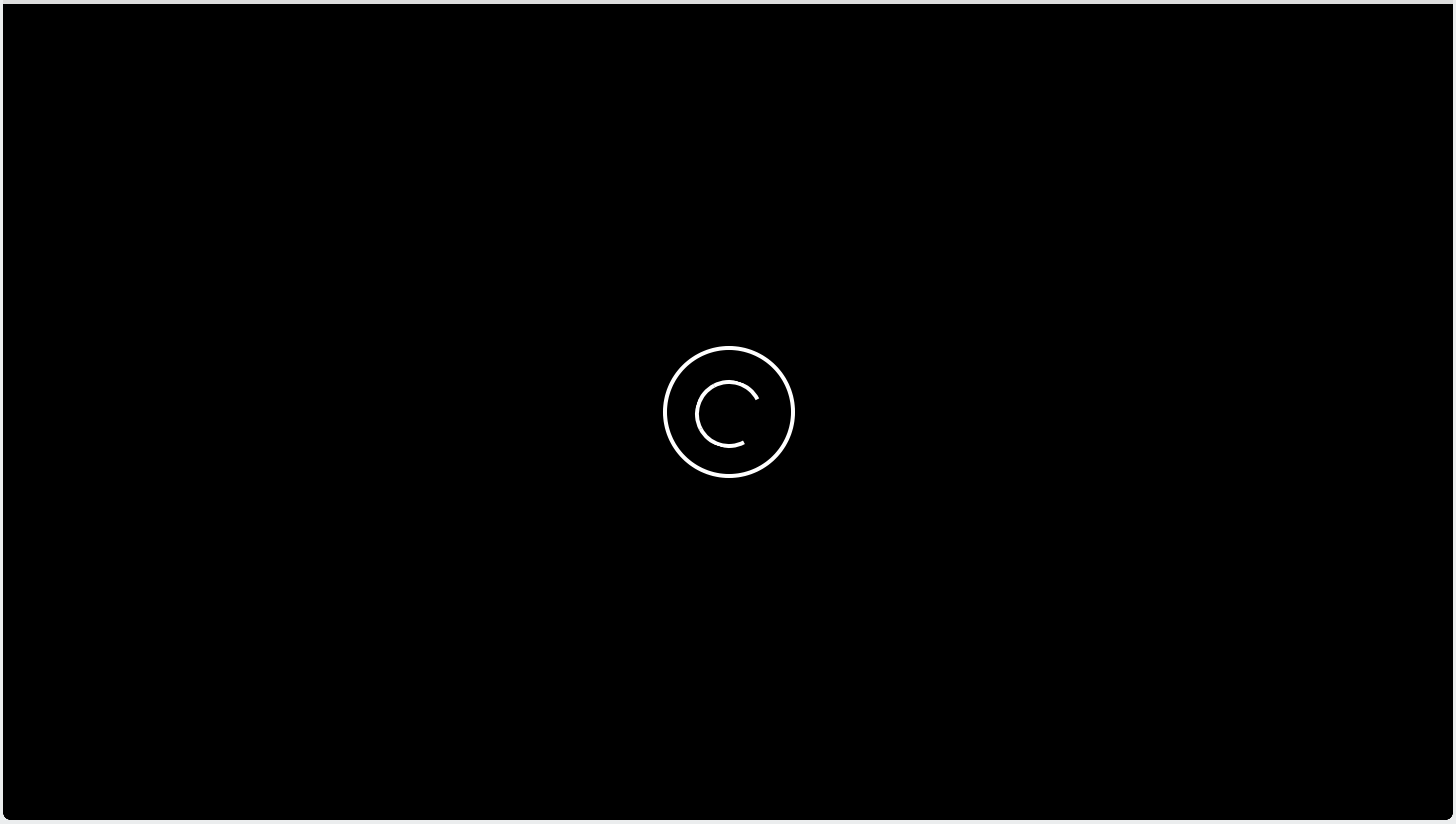
To force delete an unmerged branch, use `git branch -D branch-name`.

‹ Previous Sign in to track progress Next ›

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE



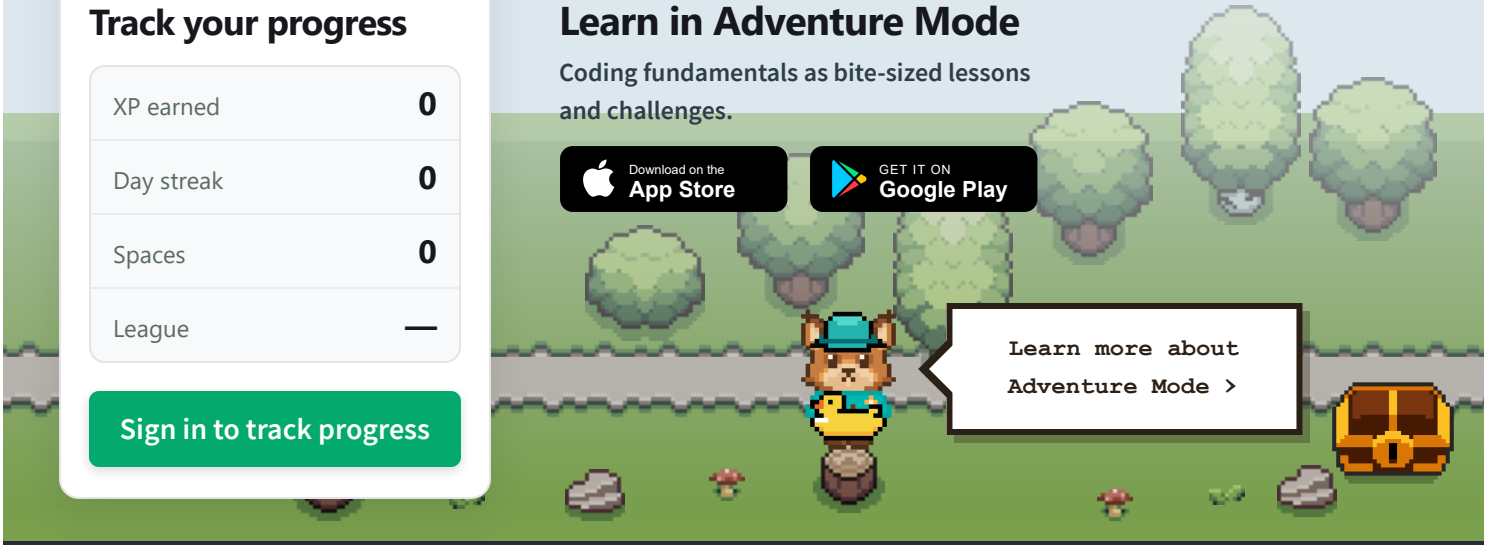
REMOVE ADS



schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

FREE ACCOUNT
Track your progress
XP earned 0
Day streak 0
Spaces 0
League —
Sign in to track progress

NEW
Learn in Adventure Mode
Coding fundamentals as bite-sized lessons and challenges.
Download on the App Store
GET IT ON Google Play
Learn more about Adventure Mode >



schools PLUS SPACES

Tutorials Exercises
W3Schools app NEW
iOS & Android
Sign In
C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE
PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

C++ C# AWS W3.CSS HOW TO BOOTSTRAP REACT MYSQL JQUE

correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.